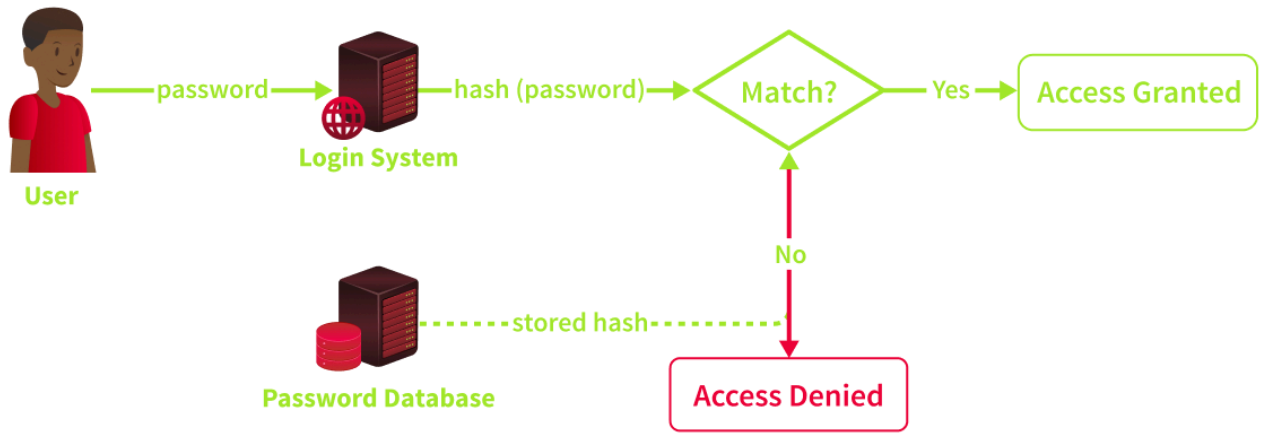
Password Cracking

How Passwords Are Stored

store a **hash** of the password rather than the password itself.

What Makes a Hash Function Useful for This



A cryptographic hash function has four properties that make it suitable for password storage:

- **One-way:** There is no way to reverse a hash back to the original input. We can only move forward by hashing a candidate and comparing.
- **Deterministic:** The same input always produces the same output. `hash("hunter2")` will always return the same value, on any machine, every time.
- **Fixed-length output:** Regardless of input length, the output is always the same size. MD5 always produces 32 hex characters, whether the input is `a` or a 10,000-character string.
- **Collision-resistant:** It should be computationally infeasible for two different inputs to produce the same hash. When an algorithm loses this property (as MD5 and SHA-1 have), it is considered broken for security purposes.

Common Hashing Algorithms

Algorithm	Output Length	Still Used for Passwords?	Notes
MD5	128 bits (32 hex chars)	No	Fast, collision-prone, widely cracked
SHA-1	160 bits (40 hex chars)	No	Faster than SHA-256, deprecated
SHA-256	256 bits (64 hex chars)	Sometimes	Better than MD5/SHA-1 but still fast
NTLM	128 bits (32 hex chars)	Yes (legacy Windows authentication)	MD4-based, used for Windows account hashes
bcrypt	~60 chars, $2*$ prefix	Yes, recommended	Deliberately slow, cost-configurable
Argon2	Variable	Yes, recommended	Modern standard, memory-hard

Salting

Even with a strong algorithm, a problem remains: if two users share the same password, their stored hashes will be identical. An attacker who pre-computes hashes for common passwords, a **rainbow table**, can look up matches instantly without running any cracking at all.

Salting solves this. A salt is a unique random string generated for each user and combined with their password before hashing:

stored_value = hash(password + salt)

Identifying Hash Types

Visual Characteristics

The fastest check is always length and format:

Hash Type	Length	Prefix / Format	Example
MD5	32 hex chars	None	5f4dcc3b5aa765d61d8327deb882cf99
SHA-1	40 hex chars	None	5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
SHA-256	64 hex chars	None	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
NTLM	32 hex chars	None	8846f7eaee8fb117ad06bdd830b7586c
bcrypt	~60 chars	$2a$, $2b$, or $2y$	$2y$12$...

The trickiest ambiguity: MD5 and NTLM are both 32 hex characters. The hash alone will not tell us which is which. Context resolves it. A hash extracted from a Windows SAM file or dumped from Active Directory is almost certainly NTLM. A hash pulled from a web application database is more likely to be MD5 or a SHA variant.

Using Hashid

apt install hashid

hashid '5f4dcc3b5aa765d61d8327deb882cf99'

```
root@tryhackme:~# hashid '5f4dcc3b5aa765d61d8327deb882cf99'
Analyzing '5f4dcc3b5aa765d61d8327deb882cf99'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin v2.x
```

hashid returns several candidates for a 32-character hex string because multiple algorithms produce output of that length. Use context to narrow it down, or try the most likely candidates in order when cracking.

Hashcat --identify

hashcat --identify '5f4dcc3b5aa765d61d8327deb882cf99'

```
root@tryhackme:~# hashcat --identify '5f4dcc3b5aa765d61d8327deb882cf99'
The following 11 hash-modes match the structure of your input hash:

      # | Name                                                       | Category
  ======+============================================================+======================================
    900 | MD4                                                        | Raw Hash
      0 | MD5                                                        | Raw Hash
     70 | md5(utf16le($pass))                                        | Raw Hash
   2600 | md5(md5($pass))                                            | Raw Hash salted and/or iterated
   3500 | md5(md5(md5($pass)))                                       | Raw Hash salted and/or iterated
   4400 | md5(sha1($pass))                                           | Raw Hash salted and/or iterated
  20900 | md5(sha1($pass).md5($pass).sha1($pass))                    | Raw Hash salted and/or iterated
   4300 | md5(strtoupper(md5($pass)))                                | Raw Hash salted and/or iterated
   1000 | NTLM                                                       | Operating System
   9900 | Radmin2                                                    | Operating System
   8600 | Lotus Notes/Domino 5                                       | Enterprise Application Software
```

Online Resources

https://crackstation.net/ : Paste a hash, and it checks against a pre-computed lookup table of billions of entries. If the plaintext has been seen before, it comes back instantly.

https://hashes.com/en/decrypt/hash: Identifies the hash type and attempts a lookup against a large database. Also accepts bulk hash submissions.

Note: On a real engagement, never submit client hashes to third-party sites. The hash may be sensitive, and any lookup creates a record of what you were cracking and when.

Hashcat Modes and John Formats

Once we have identified the algorithm, we translate it to the tool-specific format or mode number:

Algorithm	Hashcat Mode (-m)	John Format (--format=)
MD5	0	raw-md5
SHA-1	100	raw-sha1
SHA-256	1400	raw-sha256
SHA-512	1700	raw-sha512
NTLM	1000	nt
bcrypt	3200	bcrypt

These values are fixed. Running `hashcat -m 1000` always means NTLM, regardless of the hash's format. Getting the mode wrong is one of the most common causes of a crack producing no results.

Note: When hashid gives multiple candidates for a 32-character hex hash, try MD5 (mode `0`) first. If the attack completes with no results, try NTLM (mode `1000`) next.

Wordlists and Attack Strategies

If a path does not exist, use find / -name [filename] 2>/dev/null to locate it

Dictionary attacks

A dictionary attack tests a pre-built list of candidate passwords against the hash, one by one. It is the fastest first step for most cracking tasks and should always be our starting point.

The most widely used wordlist is rockyou.txt

For broader coverage, the SecLists collection

Brute-Force Attacks

A brute-force attack generates every possible character combination up to a specified length. Given unlimited time, it will always recover the password. In practice, it is rarely viable beyond 6-7 characters because the search space grows exponentially.

Pure brute force makes sense only when the search space is genuinely small: a 4-digit PIN, for instance, or a pattern so constrained that exhausting all options is fast.

Rule-Based Attacks

Rule-based attacks take an existing wordlist and apply transformations to each word, generating the mutations that people commonly use when constructing passwords:

Capitalise the first letter: password → Password
Append a number: password → password1
Add a special character: password → password!
Substitute characters: password → p@ssw0rd

Rule File	Description
best64.rule	64 highly effective mutations, good first choice
rockyou-30000.rule	30,000 rules derived from RockYou analysis
d3ad0ne.rule	Large community-built rule set
dive.rule	An extensive rule set covering a wide range of mutations
OneRuleToRuleThemAll.rule	Popular community-compiled rule set; not bundled by default, verify it exists on the system before use

Mask Attacks

A mask attack is a structured Brute-force attack in which you define the pattern of the password rather than just the character set. If we know a password follows a specific structure, for example, a word followed by a four-digit year, a mask attack generates only candidates that match that structure.

Hashcat mask syntax uses placeholders for character sets:

Placeholder	Character Set
?l	Lowercase letters (a-z)
?u	Uppercase letters (A-Z)
?d	Digits (0-9)
?s	Special characters
?a	All printable ASCII

A mask for a password like `Summer2026!` would be `?u?l?l?l?l?l?d?d?d?d?s`. This generates far fewer candidates than a full brute force of the same length.

Choosing the Right Approach

Scenario	Best approach
No information about the password	Dictionary attack with rockyou.txt
Dictionary fails, password likely mutated	Dictionary + rules (e.g., best64.rule)
Known password pattern or enforced policy	Mask attack
Short password, small character set	Brute force (constrained length only)
Target likely used company-specific terms	Custom wordlist + rules

Start with a dictionary attack because it is fast and covers the most common passwords. Move to rules if it fails. Use masks when we have reliable information about the password structure.

Cracking with John the Ripper and Hashcat

John the Ripper

basic dictionary attack :

john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt demo.txt

Flags explained:

--format=raw-md5, specifies the hash format explicitly (use the John format labels from Task 3)
--wordlist=, path to the wordlist file
demo.txt, the file containing one hash per line

Auto-detect mode (when unsure of the format):

john --wordlist=/usr/share/wordlists/rockyou.txt demo.txt

Rule-Based Attack:
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt --rules=wordlist demo2.txt

Viewing Cracked Passwords
John stores results in its potfile at /usr/local/john/run/john.pot. To display previously cracked hashes:

john --show --format=raw-md5 demo.txt

Hashcat

Hashcat is GPU-accelerated, making it significantly faster than John for most hash types.

Basic Dictionary Attack:

hashcat -m 0 -a 0 demo.txt /usr/share/wordlists/rockyou.txt

Flags explained:

-m 0, hash type (0 = MD5; use the mode numbers from Task 3)
-a 0, attack mode (0 = dictionary)
demo.txt, the file containing one hash per line
The final argument is the wordlist path

Rule-Based Attack:

hashcat -m 0 -a 0 demo2.txt /usr/share/wordlists/rockyou.txt -r
/usr/local/hashcat/rules/best64.rule

Flags explained:

-r, specifies a rule file to apply to the wordlist

Mask Attack:

hashcat -m 0 -a 3 demo3.txt '?l?l?l?l?l?l?l?l'

Flags explained:

-a 3, attack mode 3 = mask attack
The final argument is the mask pattern

Saving Output to a File

hashcat -m 0 -a 0 demo2.txt /usr/share/wordlists/rockyou.txt -o cracked.txt

Viewing Results After the Run

hashcat -m 0 demo.txt --show

Performance notes

Comparison

	John the Ripper	Hashcat
Acceleration	CPU (primarily)	GPU (primarily, CPU fallback)
Speed (MD5/SHA)	Fast	Very fast
Format detection	Good auto-detect	Explicit mode required
Non-standard formats	Excellent	Good
Rule sets	Built-in + extensible	Large file library
Best for	Quick attempts, varied formats, shadow files	Sustained attacks, GPU-accelerated cracking